第 4 课：成本函数

吴恩达《Machine Learning Specialization》· Course 1 · Week 1 · Lesson 3

承上：第 3 课我们认识了"误差"——每个点的预测值和真实值差多远。 **这一课**：我们把"所有点的误差"汇总成**一个数**，用它来判断"哪条线最好"。这个数，就叫 **成本函数（Cost Function）**。

本课主角公式（别怕，会一个字一个字拆开讲）：

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f(x^{(i)}) - y^{(i)}\right)^2$$

数学基础不好？ 公式里用到的"平方、求和 Σ、平均数"，都在旁边的《数学基础急救包》里，卡住就翻回去看。

怎么用：从头往下读，灰色代码格按 `Shift + Enter` 运行。

第 0 步：环境自检

运行下面这格，确认画图工具就绪、中文字体也设好。

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import font_manager
# ---- 选择你电脑上存在的中文字体：Windows 用微软雅黑，Mac 用苹方，Linux 用 Noto ----
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["savefig.bbox"] = "tight"    # 保存时自动裁掉多余空白、防止边缘被切
print("本次使用中文字体：", _chosen)
print(" 环境就绪，可以继续啦！")
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
 环境就绪，可以继续啦！
```

一、回顾：我们到底在找什么？

第 3 课的主角是一个直线模型：

$$f(x) = w \cdot x + b$$

`w` 和 `b` 是两个**可以拧的旋钮**。我们的目标，是**找一对最好的 (w, b)**，让这条线尽量贴着数据走。

那"贴着数据走"到底怎么判断？第 3 课我们画了"误差小竖线"：每个点，预测值和真实值差多远，就是那套房子的误差。

一句话：**"最贴合" = 所有点的误差都尽量小。**

可问题来了：**误差有一大堆，怎么比"谁大谁小"？** 我们得把它们**汇总成一个数**——数越小，线越好。这个汇总出来的数，就是今天的主角：**成本函数**。

二、从"一个误差"到"一个总成本"

我们把这件事**分成 4 小步**做，每一步都很简单：

第 1 步：先算出每个点的误差

$$\text{误差} = f(x^{(i)}) - y^{(i)} = (\text{预测值}) - (\text{真实值})$$

第 2 步：把每个误差"平方"一下

$$\left(f(x^{(i)}) - y^{(i)}\right)^2$$

第 3 步：把所有点的平方误差"全加起来"（还记得 `Σ` 吗？）

$$\sum_{i=1}^{m} \left(f(x^{(i)}) - y^{(i)}\right)^2$$

第 4 步：再除以 `2m`，把它变成一个"平均值"级别的数：

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f(x^{(i)}) - y^{(i)}\right)^2$$

看，你刚才一步步拼出来的，就是那个看起来吓人的成本函数。**它一点都不神秘，就是"误差 → 平方 → 求和 → 平均"。**

2.1 逐字拆解这个公式

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f(x^{(i)}) - y^{(i)}\right)^2$$

符号	读法	它在干嘛
`J(w, b)`	J 是 w、b 的函数	**汇总出来的总成本**，越小越好
`w, b`	两个旋钮	我们要"找"的就是它俩
`m`	样本个数	一共几套房（第 2 课学过）
`f(x⁽ⁱ⁾)`	第 i 个点的预测值	模型估出来的房价
`y⁽ⁱ⁾`	第 i 个点的真实值	真实房价
`(f(x⁽ⁱ⁾) − y⁽ⁱ⁾)²`	第 i 个点的平方误差	这一套房子错得多离谱
`Σ`	求和	把所有点的平方误差加起来
`1/(2m)`	除以 2m	求平均 + 一个小小的技术处理

用一句人话读出来：

> "把每个点的 **平方误差** 全加起来，再除以 **2m**，得到的就是 `J(w, b)`。"

2.2 两个"为什么"

为什么要把误差平方？ 两个原因：

1. **防止正负抵消**。有的点预测高了（误差 +5），有的低了（误差 −5），直接相加 = 0，好像没误差一样。一平方，负的就变正，抵消不了。
2. **让小错小罚、大错重罚**。误差 10 平方是 100，误差 2 平方才 4——偏差越大，惩罚越狠。

为什么要除以 2m？

- 除以 `m`：是为了**求平均**（不然样本越多、总和自然越大，没法公平比较）。
- 除以 `2`：纯粹是**为了后面（第 5 课求导）算起来更干净**，数学上的小技巧。先不用纠结，记住有这回事就行。

三、手把手算一个成本函数

光看公式容易晕，我们**真的动手算一遍**。为了数字好看，先用一组很小很小的玩具数据：

面积 x	房价 y
1	1
2	2
3	3

一共 `m = 3` 个点。为了简单，我们先**固定 b = 0**，模型就变成 `f(x) = w·x`（只剩一个旋钮 w）。

那成本函数简化为：

$$J(w) = \frac{1}{2 \times 3} \sum_{i=1}^{3} (wx_i - y_i)^2$$

下面让代码替我们把 w 取不同值时 J 是多少，一起算出来：

In [2]:

```
# 玩具数据：3 个点（面积，房价）
x = np.array([1, 2, 3])
y = np.array([1, 2, 3])

def cost(w):
    # b=0 时的成本函数：J(w) = (1/(2m)) * Σ (w*x_i - y_i)^2
    # 注意：这里用 len(x) 而不是外部的 m，函数才不会受别处同名变量的影响
    errors = w * x - y                          # 每个点的误差：预测 - 真实
    return (1 / (2*len(x))) * np.sum(errors**2)   # 平方 → 求和 → 除以 2m

print("  w    J(w)")
for w in [0.0, 0.5, 1.0, 1.5, 2.0]:
    print(f"{w:>4}   {cost(w):.3f}")
```

```
  w     J(w)
0.0   2.333
0.5   0.583
1.0   0.000
1.5   0.583
2.0   2.333
```

看结果：

- `w = 1.0` 时，**J = 0** —— 因为 f(x)=1·x 正好完美预测（预测=真实），误差全为 0。
- `w = 0.5` 或 `w = 1.5`，J 都变大了。
- `w = 0` 或 `w = 2`，偏离更远，J 更大。

结论：J 越小，说明这条线越好。 我们想要的，就是让 J 尽量小的那个 w。

（聪明的你可能发现了：w 太大或太小，J 都会变大——**J 关于 w 像一个"山谷"**，谷底就是最好的 w。下面画出来看看。）

四、亲眼看成本函数

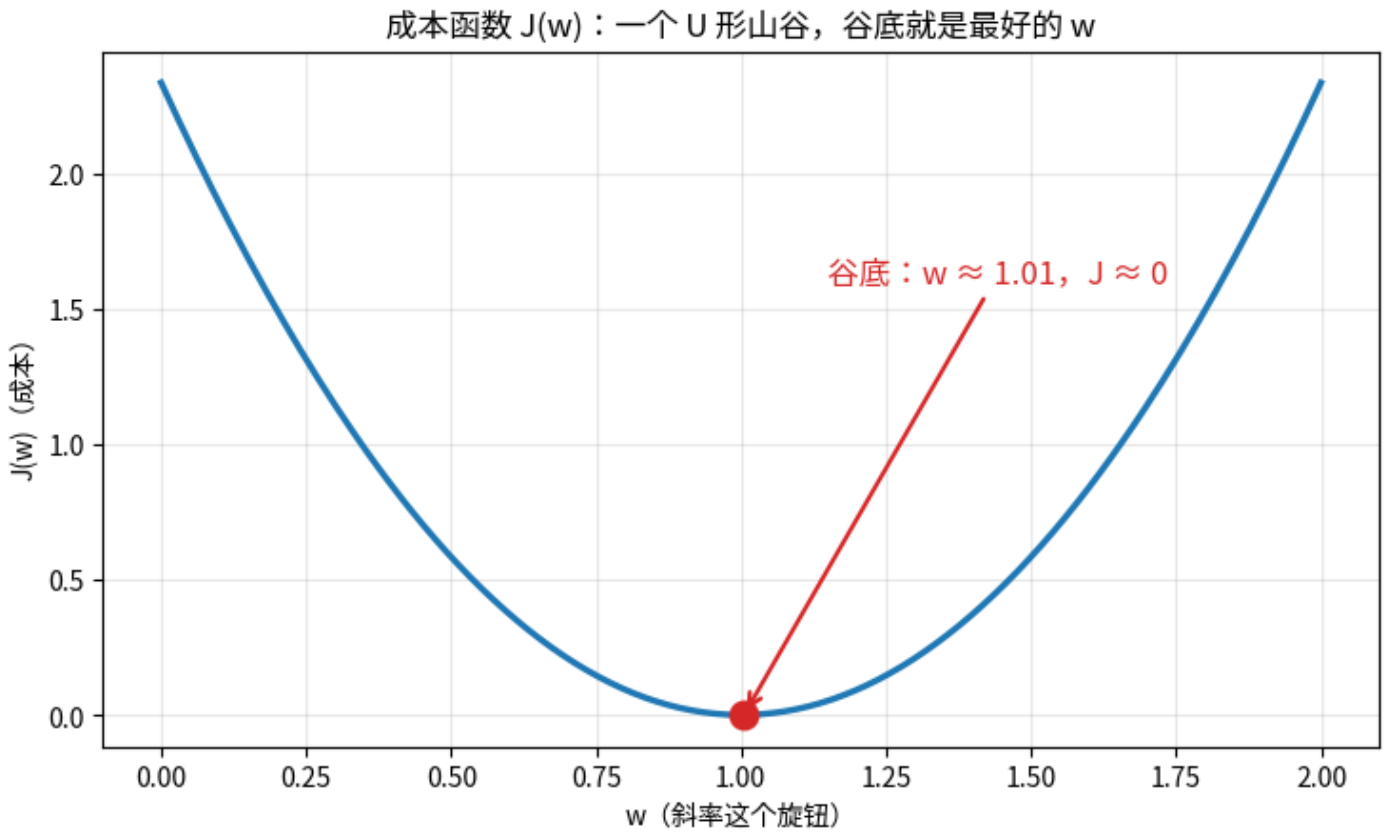
4.1 先看 J 随 w 变化（固定 b）

把上面玩具数据的 J(w) 画成曲线：

In [3]:

```
ws = np.linspace(0, 2, 200)          # 让 w 从 0 变到 2
Js = [cost(w) for w in ws]            # 每个 w 对应的成本

plt.figure(figsize=(7.5, 4.6))
plt.plot(ws, Js, color="tab:blue", linewidth=2.4)
w_best = ws[int(np.argmin(Js))]
plt.scatter([w_best], [cost(w_best)], color="tab:red", s=110, zorder=3)
plt.annotate(f"谷底：w ≈ {w_best:.2f}，J ≈ 0",
             xy=(w_best, 0), xytext=(1.15, 1.6), color="tab:red", fontsize=12,
             arrowprops=dict(arrowstyle="->", color="tab:red", lw=1.6))
plt.xlabel("w（斜率这个旋钮）")
plt.ylabel("J(w)（成本）")
plt.title("成本函数 J(w)：一个 U 形山谷，谷底就是最好的 w")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



看：曲线像一个 **U 形山谷**。两头（w 很小或很大）J 都高，中间有个**谷底**——谷底位置就是"最好的 w"。

> **记住这张图，它就是整门课的核心画面**：训练模型 = **下山，走到谷底**。（第 5 课"梯度下降"就是教机器怎么自动"走"到谷底。）

4.2 再看真实的 J(w, b)：一个"碗"

刚才我们固定了 b，只有一个旋钮，所以是一维的"山谷"。现在放开 **w 和 b 两个旋钮**，用第 2、3 课那份**真实房价数据**，成本函数就变成一个**三维的"碗"**。

三维不好画，我们画它的**俯视图**（叫**等高线图**，就像地图上的海拔线）：线越往里，J 越小。

In [4]:

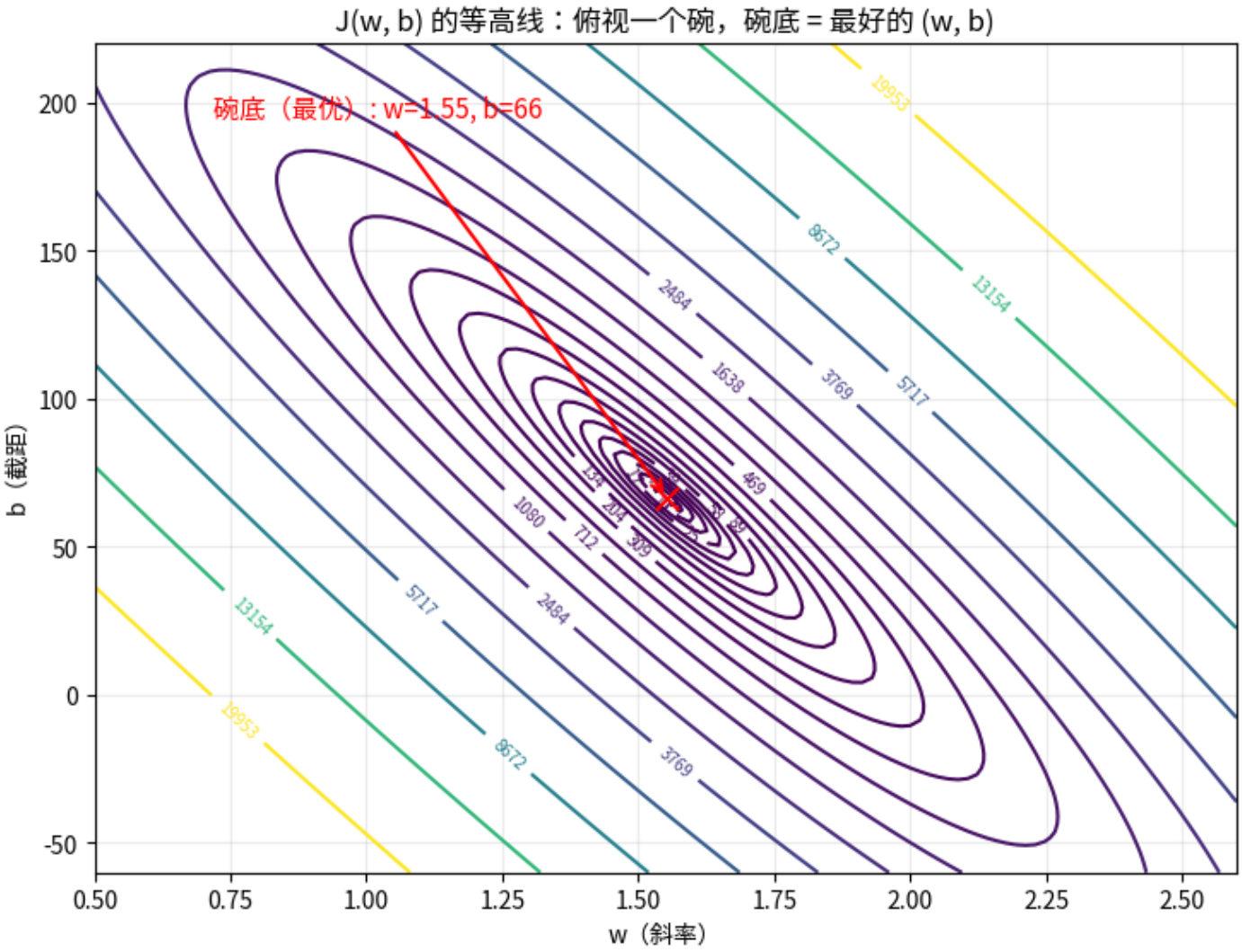
```python
# 用真实房价数据（和第 2、3 课同一份）
x_house = np.array([50, 75, 100, 125, 150, 175, 200, 225, 250])
y_house = np.array([150, 180, 220, 260, 295, 340, 375, 420, 455])

def cost_wb(w, b):                    # 完整的 J(w, b)
    # 同样用 len(x_house)，不依赖外部变量
    pred = w * x_house + b
    return (1/(2*len(x_house))) * np.sum((pred - y_house)**2)

ws = np.linspace(0.5, 2.6, 120)
bs = np.linspace(-60, 220, 120)
W, B = np.meshgrid(ws, bs)
J = np.array([[cost_wb(w, b) for w in ws] for b in bs])

plt.figure(figsize=(7.5, 5.8))
cs = plt.contour(W, B, J, levels=np.logspace(0.5, 4.3, 22), cmap="viridis")
plt.clabel(cs, inline=True, fontsize=7, fmt="%.0f")
# 标出真正的碗底：最小二乘的精确解
w_star, b_star = np.polyfit(x_house, y_house, 1)
plt.scatter([w_star], [b_star], color="red", s=90, marker="x", zorder=5)
plt.annotate(f"碗底（最优）: w={w_star:.2f}, b={b_star:.0f}",
             xy=(w_star, b_star), xytext=(0.72, 195), color="red", fontsize=11,
             arrowprops=dict(arrowstyle="->", color="red", lw=1.5))
print(f"最小二乘最优解: w = {w_star:.3f}, b = {b_star:.3f}")
plt.xlabel("w（斜率）")
plt.ylabel("b（截距）")
plt.title("J(w, b) 的等高线：俯视一个碗，碗底 = 最好的 (w, b)")
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
最小二乘最优解: w = 1.553, b = 66.444
```



看：一圈一圈的等高线，**越往里 J 越小**，正中间那个红叉（碗底）就是让 J 最小的 **w 和 b**——也就是**最好的那条线**。

> 到这里你已经抓住了机器学习的**全部工作流程**：
>
> 1. 用 `f(x)=wx+b` 当模型；
> 2. 用 `J(w,b)` 量"这条线有多差"；
> 3. **找一对 (w, b)，让 J 最小** → 就得到了最好的线。
>
> **第 3 步——怎么找？** 下一课见分晓。

五、小结

1. **成本函数** `J(w, b)`：把**所有点的平方误差**加起来、再除以 `2m` 得到的**一个数**，用来衡量"这条线有多差"。
2. **越小越好**：J 越小，线越贴合数据。
3. **为什么平方**：防正负抵消 + 重罚大误差。**为什么 1/2m**：求平均 + 方便后面求导。
4. **它的样子**：对 w（固定 b）是一条 **U 形山谷**；对 (w, b) 是一个三维的**碗**。**谷底/碗底 = 最好的参数**。
5. **一句话：训练模型 = 找到让 J 最小的那对 (w, b)。**

六、反转课堂·课后作业

1. 用你自己的话说：**成本函数 J 到底在量什么**？为什么它越小越好？
2. 用第 3 节的玩具数据（x=[1,2,3], y=[1,2,3], b=0）：**当 w=3 时**，三个点的误差分别是多少？平方和是多少？J 大约是多少？
3. 看图回答：在"U 形山谷"那张图里，**为什么 w 比谷底小很多、或者大很多时，J 都会变大**？

下一课（第 5 课）：梯度下降（Gradient Descent）——教机器怎么自动"走"到碗底，找到最好的 w 和 b。

> 编号说明：第 5 课是 **Lesson 4：梯度下降**。